

ISE 612: Final Project

Implementation of Decoded Quantum Interferometry Algorithm

Onkar V. Apte

December 17th, 2024

Contents

1	Introduction	2
2	Overview of Decoded Quantum Interferometry	2
3	Problem definition	3
3.1	max-LINSAT	3
3.2	max-XORSAT	3
3.3	MaxCut as max-XORSAT special case	3
4	DQI for max-XORSAT	4
4.1	DQI Quantum State	4
4.2	Algorithm	5
4.2.1	Step 1	5
4.2.2	Step 2	5
4.2.3	Step 3	6
4.2.4	Step 4	6
4.2.5	Step 5	6
4.2.6	Step 6	6
5	Results [Unfinished]	7
A	Explicit walk-through of example problem	8
A.1	Step 1	8
A.2	Step 2	9
A.3	Step 3	9
A.4	Step 4	9
A.5	Step 5	10
A.6	Step 6	10
B	Intuition behind why this algorithm works [Unfinished]	11

Abstract

I tried to implement the DQI algorithm. I spent well over 30 hours. Most of this time was spent thoroughly understanding the algorithm, as it is highly complex and required multiple readings of the paper, as well as trying things out with pen and paper, to grasp its concepts not only in a mechanical way but also in an intuitive manner. However, due to the time constraint, I could not manage to get the implementation for this working. I am submitting only the report I have prepared, and I hope this will be sufficient. However, if a programming part is mandatory for the final project, I can pivot to the *Optimization with the Adiabatic Theorem* project from the provided list and likely complete it within a couple of hours, if given an extension. Please let me know if this alternative would be acceptable. However, I sincerely hope this submission meets the requirements.

1 Introduction

For the final project of course ISE 612, we implement the Decoded Quantum Interferometry (DQI) algorithm as presented in [1] to solve the max-XORSAT problem. In Section 2, we provide an overview of the DQI algorithm. In Section 3, we define the problems DQI can solve. Section 4 gives the algorithm for max-XORSAT. In Section 5, we talk about results for the specific instance of max-XORSAT we choose to implement. In Appendix A, we analytically follow the DQI algorithm and see how it works for the problem instance we choose. In Appendix B, we give an intuition about the algorithm. In contrast to the mechanistic approach of the algorithm description from 4, here we try to give an idea of how you could have come up with an algorithm.

2 Overview of Decoded Quantum Interferometry

DQI algorithm reduces some classical optimization problems to problems of decoding classical error correction codes. DQI achieves this by exploiting structure in the Fourier spectrum of the objective function.

The general optimization problem that DQI is applicable to is max-LINSAT as defined in Section 3.1. If we restrict max-LINSAT to $\mathbb{F}_2$, we obtain max-XORSAT problems, as discussed in Section 3.2. If we add some more restrictions to max-XORSAT, we get the problem of MaxCut. All of these problems are hard.

Although DQI is applicable to max-LINSAT problems, there are only two examples of optimization problems where DQI is effective. The first one is Optimal Polynomial Intersection (OPI), a special case of max-LINSAT, where DQI outperforms all known classical algorithms. Second is sparse max-XORSAT, where DQI outperforms simulated annealing, but specialized classical algorithms for these problems can still outperform DQI. For OPI, DQI efficiently achieves a better approximation ratio than any polynomial-time classical algorithm known to us, realizing an apparent exponential speedup. It is natural to ask why achieving a better approximation ratio allows authors to claim exponential speedup. Classical algorithms in context give an approximation ratio of 0.55, whereas DQI gives 0.7179. It is not always true that improving the approximation ratio corresponds to getting an exponential quantum speedup. However, for OPI, once we exceed the approximation ratio of 0.55, the only classical algorithms that we know of are slightly refined versions of exhaustive search, which take exponential time. Thus, authors claim *apparent* exponential speedup.

3 Problem definition

In this section, we will define the problems DQI is applicable to.

3.1 max-LINSAT

Definition 1 Let $\mathbb{F}$ be a finite field. For $i \in (1, 2, \dots, m)$, let

$$f_i : \mathbb{F} \rightarrow \{+1, -1\}, \quad (1)$$

be arbitrary functions. Given a matrix $B \in \mathbb{F}^{m \times n}$, max-LINSAT problem is to find $\vec{x} \in \mathbb{F}^n$ maximizing the objective function

$$f(\vec{x}) = \sum_{i=1}^m f_i \left(\sum_{j=1}^n B_{ij} x_j \right). \quad (2)$$

3.2 max-XORSAT

Definition 2 Given $B \in \mathbb{F}_2^{m \times n}$ and $\vec{v} \in \mathbb{F}_2^m$ with $m > n$, the max-XORSAT problem is to find $\vec{x} \in \mathbb{F}_2^n$ such that it satisfies as many as possible linear mod-2 equations out of m , given by $B\vec{x} = \vec{v}$. The objective function

$$f : \mathbb{F}_2^n \rightarrow \mathbb{Z}, \quad (3)$$

can be taken as number of equations satisfied minus the number of equations unsatisfied, which is given as

$$f(\vec{x}) = \sum_{i=1}^m (-1)^{v_i + \vec{b}_i \cdot \vec{x}}, \quad (4)$$

where $\vec{b}_i$ is the i^{th} row of B .

3.3 MaxCut as max-XORSAT special case

Now, we put the restriction that each column of B has exactly non-zero entries (and thus two entries of ones) on the max-XORSAT. This is just MaxCut problem where $\vec{v}$ is vector of all ones.

Consider the MaxCut problem from Homework 6.

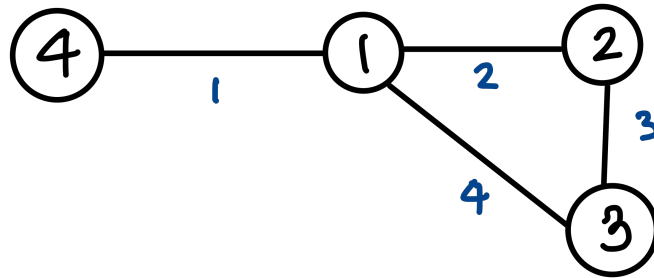


Figure 1: MaxCut instance taken from Homework 6. The blue numbers represent edge labels. For example, Edge 1 connects Nodes 4 and 1.

We can define max-XORSAT instance for this by setting B follows:

$$B = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}, \quad (5)$$

and setting $\vec{v}$ as $\vec{v} = \vec{1}$. The way we interpret B is i^{th} column of B represents Edge i , and non-zero entries represent the nodes to which Edge i connects. For example, the first column represents Edge 1, which connects Node 1 and 4, as the 1st and 4th entry of the first column is non-zero. Hence, we will have, in general, n edges and m nodes.

4 DQI for max-XORSAT

We will discuss the DQI algorithm for max-XORSAT in two parts. In Section 4.1, we will see the goal state we are trying to prepare, measuring which should give an approximate solution to our problem with very high probability. Then in Section 4.2, we will see how we can actually prepare that state efficiently.

4.1 DQI Quantum State

Our goal is to prepare the following state, referred to as, DQI state.

$$|P(f)\rangle = \sum_{\vec{x} \in \mathbb{F}_2^n} P(f(\vec{x})) |\vec{x}\rangle, \quad (6)$$

where P is a polynomial of degree- l .

$$P(f) = \sum_{k=0}^l \alpha_k f^k. \quad (7)$$

The objective function will have the form $f = f_1 + \dots + f_m$. But $\forall i, f_i \in \{+1, -1\} \implies f_i^2 = 1$. Therefore, we can rewrite $P(f)$ as

$$P(f) = \sum_{k=0}^l u_k P^{(k)}(f_1, \dots, f_m), \quad (8)$$

where $P^{(k)}(f_1, \dots, f_m)$ is the degree- k polynomial, which is the unweighted sum of all ${}^m C_k$ products of k distinct factors from $f_1, \dots, f_m$. Now, we will write the DQI state as a linear combination of these $|P^{(k)}\rangle$, where

$$|P^{(k)}\rangle = \frac{1}{\sqrt{2^n \cdot {}^m C_k}} \sum_{\vec{x} \in \mathbb{F}_2^n} P^{(k)}(f_1(\vec{b}_1 \cdot \vec{x}), \dots, f_m(\vec{b}_m \cdot \vec{x})) |\vec{x}\rangle. \quad (9)$$

But from Eq.(4), we know that $f_i(\vec{b}_i \cdot \vec{x}) = (-1)^{v_i} (-1)^{\vec{b}_i \cdot \vec{x}}$. Hence we can re-write $P^{(k)}$ as

$$P^{(k)}(f_1, \dots, f_m) = \sum_{\substack{\vec{y} \in \mathbb{F}_2^n \\ |\vec{y}|_H = k}} (-1)^{\vec{v} \cdot \vec{y}} (-1)^{(B^T \vec{y}) \cdot \vec{x}}, \quad (10)$$

where $|\vec{y}|_H$ represents hamming weight of $\vec{y}$, i.e., number of ones in it. From this, we can calculate that Hadamard transform of $|P^{(k)}\rangle$ can be written as

$$|\tilde{P}^{(k)}\rangle = H^{\otimes n} |P^{(k)}\rangle = \frac{1}{\sqrt{n C_k}} \sum_{\substack{\vec{y} \in \mathbb{F}_2^n \\ |\vec{y}|_H = k}} (-1)^{\vec{v} \cdot \vec{y}} |B^T \vec{y}\rangle . \quad (11)$$

Now, using some linear block code theory, we can say that under certain conditions, $|\tilde{P}^{(0)}\rangle, |\tilde{P}^{(1)}\rangle, \dots, |\tilde{P}^{(l)}\rangle$ forms an orthonormal set and so do $|P^{(0)}\rangle, |P^{(1)}\rangle, \dots, |P^{(l)}\rangle$.

Therefore, when those conditions are satisfied, we can express Eq.(6) as

$$|P(f)\rangle = \sum_{k=0}^l w_k |P^{(k)}\rangle , \quad (12)$$

where

$$w_k = u_k \sqrt{2^n \cdot n C_k} , \quad (13)$$

such that $\langle P(f)|P(f)\rangle = \|\vec{w}\|^2$.

4.2 Algorithm

To implement the DQQI algorithm preparing the DQI state $|P(f)\rangle$, we require three quantum registers.

1. $[W]$ Weight register with $\lceil \log_2(l) \rceil$ qubits
2. $[E]$ Error register with m qubits
3. $[S]$ Syndrome register with n qubits

4.2.1 Step 1

Initialize the weight register in the state

$$|0\rangle^{\otimes \lceil \log_2(l) \rceil} \rightarrow \sum_{k=0}^l w_k |k\rangle , \quad (14)$$

where the choice of $\vec{w} \in \mathbb{R}^{l+1}$ which maximizes the expected number of satisfied constraints can be obtained by solving for the principal eigenvector of an $(l+1) \times (l+1)$ matrix, described in Section 6 of [1].

This can be done using amplitude encoding with $O(l \log(l))$ gates.

4.2.2 Step 2

For given k in weight register, prepare a uniform superposition of all bistring with hamming weight k in the error register.

$$\sum_{k=0}^l w_k |k\rangle_W \rightarrow \sum_{k=0}^l w_k \frac{1}{\sqrt{n C_k}} \sum_{\substack{\vec{y} \in \mathbb{F}_2^m \\ |\vec{y}|_H = k}} |\vec{y}\rangle_E . \quad (15)$$

This can be done using $O(lm)$ gates as given in [2].

4.2.3 Step 3

We apply the phase $(-1)^{\vec{v} \cdot \vec{y}}$ using Z -gate on each qubit where $v_i = 1$.

$$\sum_{k=0}^l w_k |k\rangle_W \frac{1}{\sqrt{nC_k}} \sum_{\substack{\vec{y} \in \mathbb{F}_2^m \\ |\vec{y}|_H = k}} |\vec{y}\rangle_E \rightarrow \sum_{k=0}^l w_k |k\rangle_W \frac{1}{\sqrt{nC_k}} \sum_{\substack{\vec{y} \in \mathbb{F}_2^m \\ |\vec{y}|_H = k}} (-1)^{\vec{v} \cdot \vec{y}} |\vec{y}\rangle_E. \quad (16)$$

This can be done using $O(m)$ gates.

4.2.4 Step 4

Now, we reversibly compute $B^T \vec{y}$ into the syndrome register by matrix-vector multiplication.

$$\sum_{k=0}^l w_k |k\rangle_W \frac{1}{\sqrt{nC_k}} \sum_{\substack{\vec{y} \in \mathbb{F}_2^m \\ |\vec{y}|_H = k}} (-1)^{\vec{v} \cdot \vec{y}} |\vec{y}\rangle_E \rightarrow \sum_{k=0}^l w_k |k\rangle_W \frac{1}{\sqrt{nC_k}} \sum_{\substack{\vec{y} \in \mathbb{F}_2^m \\ |\vec{y}|_H = k}} (-1)^{\vec{v} \cdot \vec{y}} |\vec{y}\rangle_E |B^T \vec{y}\rangle_S. \quad (17)$$

4.2.5 Step 5

We are almost there. This is the crux of this algorithm. All we need to do now to obtain $|\tilde{P}^{(k)}\rangle$ state is uncompute the weight register and error register. We need to use the content of syndrome register $B^T \vec{y}$ and compute $\vec{y}$ from it. Once we know $\vec{y}$, k is just the hamming weight of $\vec{y}$ and hence, we can uncompute both. This task is exactly same as decoding error of a certain linear error correcting code which has parity check matrix B^T , syndrome is $B^T \vec{y}$ and unknown error which caused this syndrome is $\vec{y}$. Once we perform this decoding of errors on the superposition, we can get the desired state in the syndrome register.

$$\sum_{k=0}^l w_k |k\rangle_W \frac{1}{\sqrt{nC_k}} \sum_{\substack{\vec{y} \in \mathbb{F}_2^m \\ |\vec{y}|_H = k}} (-1)^{\vec{v} \cdot \vec{y}} |\vec{y}\rangle_E |B^T \vec{y}\rangle_S \rightarrow \sum_{k=0}^l w_k \frac{1}{\sqrt{nC_k}} \sum_{\substack{\vec{y} \in \mathbb{F}_2^m \\ |\vec{y}|_H = k}} (-1)^{\vec{v} \cdot \vec{y}} |B^T \vec{y}\rangle_S. \quad (18)$$

The state in the syndrom register is nothing but

$$\sum_{k=0}^l w_k \frac{1}{\sqrt{nC_k}} \sum_{\substack{\vec{y} \in \mathbb{F}_2^m \\ |\vec{y}|_H = k}} (-1)^{\vec{v} \cdot \vec{y}} |B^T \vec{y}\rangle_S = \sum_{k=0}^l w_k |\tilde{P}^{(k)}\rangle.$$

Complexity of this step depends on the particular classical code. The classical code is determined by B^T , and B is determined by the problem instance. For example. Sparse max-XORSAT gives rise to LDPC codes in matrix B , for which we have efficient decoding algorithms.

4.2.6 Step 6

Now, we take the Hadamard transform by applying $H^{\otimes n}$ as discussed in Section 6 and obtain the desired DQI state.

$$\sum_{k=0}^l w_k |\tilde{P}^{(k)}\rangle \rightarrow \sum_{k=0}^l w_k |P^{(k)}\rangle = |P(f)\rangle. \quad (19)$$

This concludes our algorithm.

5 Results **[Unfinished]**

Even though DQI does not offer advantage for max-XORSAT, we thought it was easier to implement, specifically MaxCut, using DQI. In this section, we will define the specific MaxCut instance we solve and compare it with classical sampling random strings and compare the performance, assuming we can get the implementation working on time.

We will compare classically sampling $(f(x))^2$ with DQI, as that would be a fair comparison since for the following problem, $l = 2$, giving us 2nd degree polynomial. We expect that DQI would be more biased towards strings with higher function value than strings we will get from classical sampling of $(f(x))^2$.

In Section A, we also follow the DQI algorithm analytically and show it works for this specific problem.

We will try to solve the following graph for MaxCut problem.

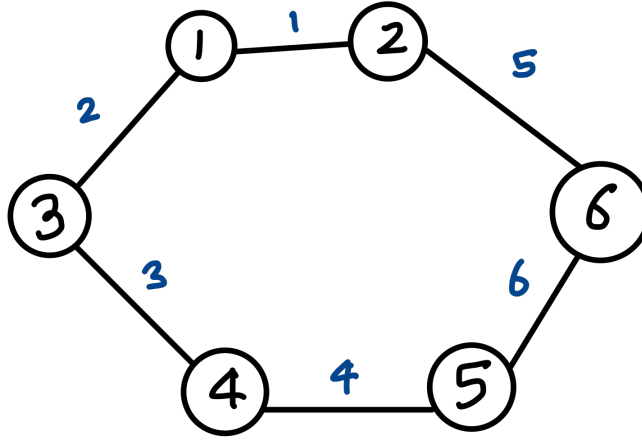


Figure 2: MaxCut instance which we try to solve. The blue numbers represent edge labels. For example, Edge 5 connects Nodes 2 and 6.

From Fig. 2, we can write down the problem inputs B and $\vec{v}$.

$$B = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix}, \quad \vec{v} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}. \quad (20)$$

Unfortunately, due to time constraint, we could not produce result in time, as we could not get the program implementation ready.

References

- [1] Stephen P. Jordan et al. *Optimization by Decoded Quantum Interferometry*. 2024. arXiv: [2408.08292](https://arxiv.org/abs/2408.08292) [quant-ph]. URL: <https://arxiv.org/abs/2408.08292>.

- [2] Andreas Bärtschi and Stephan Eidenbenz. “Deterministic Preparation of Dicke States”. In: *Fundamentals of Computation Theory*. Springer International Publishing, 2019, pp. 126–139. ISBN: 9783030250270. DOI: [10.1007/978-3-030-25027-0_9](https://doi.org/10.1007/978-3-030-25027-0_9). URL: http://dx.doi.org/10.1007/978-3-030-25027-0_9.

A Explicit walk-through of example problem

In this section, we will analytically follow the DQI and see the results. As discussed in Section 5, we have problem input

$$B = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix}, \quad \vec{v} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}.$$

We can compute the distance of error correcting code using B and we can conclude that it can correct upto two errors. Hence $l = 2$. So we have

$$\begin{aligned} m &= 6, \\ n &= 6, \\ l &= 2. \end{aligned}$$

Our objective function is

$$f(\vec{x}) = \sum_{i=1}^m (-1)^{v_i + \vec{b}_i \cdot \vec{x}},$$

where $\vec{b}_i$ is the i^{th} row of B . From $B\vec{x} = \vec{v}$, we get six constraints,

$$\begin{aligned} x_1 \oplus x_2 &= 1, \\ x_1 \oplus x_5 &= 1, \\ x_2 \oplus x_3 &= 1, \\ x_3 \oplus x_4 &= 1, \\ x_4 \oplus x_6 &= 1, \\ x_5 \oplus x_6 &= 1. \end{aligned}$$

Constraint $x_i \oplus x_j = 1$ will be satisfied if nodes i and j are on the different sides of the cut.

A.1 Step 1

Computing the optimal choice of $\vec{w}$ gives us

$$\vec{w} = \begin{bmatrix} 0 \\ \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}.$$

Therefore, we start by preparing the state

$$\frac{1}{\sqrt{2}} |01\rangle + \frac{1}{\sqrt{2}} |10\rangle$$

A.2 Step 2

Now depending on the state in register W , we prepare equal superposition of states having hamming weight k .

$$\begin{aligned} & \frac{1}{\sqrt{2}} |01\rangle \frac{1}{\sqrt{6}} \left(\underbrace{|100000\rangle + |010000\rangle + |001000\rangle + |000100\rangle + |000010\rangle + |000001\rangle}_{\text{all states with hamming weight 1}} \right) \\ & + \frac{1}{\sqrt{2}} |10\rangle \frac{1}{\sqrt{15}} \left(\underbrace{|110000\rangle + |101000\rangle + |100100\rangle + |100010\rangle + |100001\rangle + |011000\rangle + |010100\rangle + \right. \\ & \left. |010010\rangle + |010001\rangle + |001100\rangle + |001010\rangle + |001001\rangle + |000110\rangle + |000101\rangle + |000011\rangle}_{\text{all states with hamming weight 2}} \right) \end{aligned}$$

From here onward, we will write decimal numbers instead of writing binary strings. The state above is

$$\begin{aligned} & \frac{1}{\sqrt{2}} |1\rangle \frac{1}{\sqrt{6}} \left(|32\rangle + |16\rangle + |8\rangle + |4\rangle + |2\rangle + |1\rangle \right) \\ & + \frac{1}{\sqrt{2}} |2\rangle \frac{1}{\sqrt{15}} \left(|48\rangle + |40\rangle + |36\rangle + |34\rangle + |33\rangle + |24\rangle + |20\rangle + |18\rangle \right. \\ & \left. + |17\rangle + |12\rangle + |10\rangle + |9\rangle + |6\rangle + |5\rangle + |3\rangle \right) \end{aligned}$$

A.3 Step 3

Now we apply phase depending on the value of $\vec{v} \cdot \vec{y}$.

$$\begin{aligned} & \frac{1}{\sqrt{2}} |1\rangle \frac{1}{\sqrt{6}} \left(-|32\rangle - |16\rangle - |8\rangle - |4\rangle - |2\rangle - |1\rangle \right) \\ & + \frac{1}{\sqrt{2}} |2\rangle \frac{1}{\sqrt{15}} \left(|48\rangle + |40\rangle + |36\rangle + |34\rangle + |33\rangle + |24\rangle + |20\rangle + |18\rangle \right. \\ & \left. + |17\rangle + |12\rangle + |10\rangle + |9\rangle + |6\rangle + |5\rangle + |3\rangle \right) \end{aligned}$$

A.4 Step 4

Now we compute the $B^T \vec{y}$ in the syndrome register.

$$\begin{aligned} & \frac{1}{\sqrt{2}} |1\rangle \frac{1}{\sqrt{6}} \left(-|32\rangle |48\rangle - |16\rangle |34\rangle - |8\rangle |24\rangle - |4\rangle |12\rangle - |2\rangle |5\rangle - |1\rangle |3\rangle \right) \\ & + \frac{1}{\sqrt{2}} |2\rangle \frac{1}{\sqrt{15}} \left(|48\rangle |18\rangle + |40\rangle |40\rangle + |36\rangle |60\rangle + |34\rangle |53\rangle + |33\rangle |51\rangle + |24\rangle |58\rangle + |20\rangle |46\rangle + |18\rangle |39\rangle \right. \\ & \left. + |17\rangle |33\rangle + |12\rangle |20\rangle + |10\rangle |29\rangle + |9\rangle |27\rangle + |6\rangle |9\rangle + |5\rangle |15\rangle + |3\rangle |6\rangle \right) \end{aligned}$$

A.5 Step 5

Now we do the uncomputation.

$$\begin{aligned} & \frac{1}{\sqrt{12}} \left(-|48\rangle - |34\rangle - |24\rangle - |12\rangle - |5\rangle - |3\rangle \right) \\ & + \frac{1}{\sqrt{30}} \left(|18\rangle + |40\rangle + |60\rangle + |53\rangle + |51\rangle + |58\rangle + |46\rangle + |39\rangle \right. \\ & \left. + |33\rangle + |20\rangle + |29\rangle + |27\rangle + |9\rangle + |15\rangle + |6\rangle \right) \end{aligned}$$

A.6 Step 6

Now we apply $H^{\otimes 6}$ to the state we obtained in Step 5. Doing this analytically is really complicated. We use computer to program this and get the following statevector, where number in the bracket is the amplitude and binary string after it is the basis state with that amplitude.

```
(0.12582024749461906+0j) 000000
(-0.09499055687808512+0j) 000001
(-0.09499055687808511+0j) 000010
(-0.09499055687808511+0j) 000011
(-0.0949905568780851+0j) 000100
(-0.0949905568780851+0j) 000101
(0.049347010419321304+0j) 000110
(-0.09499055687808511+0j) 000111
(-0.0949905568780851+0j) 001000
(0.0493470104193213+0j) 001001
(0.049347010419321276+0j) 001010
(0.049347010419321276+0j) 001011
(-0.0949905568780851+0j) 001100
(-0.09499055687808511+0j) 001101
(0.04934701041932129+0j) 001110
(-0.09499055687808512+0j) 001111
(-0.0949905568780851+0j) 010000
(0.04934701041932129+0j) 010001
(0.04934701041932126+0j) 010010
(0.04934701041932127+0j) 010011
(0.049347010419321276+0j) 010100
(0.049347010419321276+0j) 010101
(0.5588329493868383+0j) 010110
(0.04934701041932127+0j) 010111
(-0.09499055687808511+0j) 011000
(0.04934701041932129+0j) 011001
(0.049347010419321276+0j) 011010
(0.049347010419321276+0j) 011011
(-0.09499055687808511+0j) 011100
(-0.09499055687808511+0j) 011101
(0.04934701041932129+0j) 011110
```

(-0.0949905568780851+0j) 011111
 (-0.0949905568780851+0j) 100000
 (0.04934701041932129+0j) 100001
 (-0.0949905568780851+0j) 100010
 (-0.0949905568780851+0j) 100011
 (0.049347010419321276+0j) 100100
 (0.049347010419321276+0j) 100101
 (0.04934701041932129+0j) 100110
 (-0.0949905568780851+0j) 100111
 (0.04934701041932127+0j) 101000
(0.5588329493868383+0j) 101001
 (0.049347010419321276+0j) 101010
 (0.049347010419321276+0j) 101011
 (0.04934701041932127+0j) 101100
 (0.04934701041932126+0j) 101101
 (0.04934701041932129+0j) 101110
 (-0.0949905568780851+0j) 101111
 (-0.09499055687808512+0j) 110000
 (0.04934701041932129+0j) 110001
 (-0.0949905568780851+0j) 110010
 (-0.0949905568780851+0j) 110011
 (0.049347010419321276+0j) 110100
 (0.049347010419321276+0j) 110101
 (0.0493470104193213+0j) 110110
 (-0.0949905568780851+0j) 110111
 (-0.0949905568780851+0j) 111000
 (0.049347010419321304+0j) 111001
 (-0.0949905568780851+0j) 111010
 (-0.0949905568780851+0j) 111011
 (-0.0949905568780851+0j) 111100
 (-0.0949905568780851+0j) 111101
 (-0.09499055687808512+0j) 111110
 (0.12582024749461906+0j) 111111

We can observe that two states $|101001\rangle$ and $|010110\rangle$ (highlighted above with bold) have amplitude 0.55 each, which is significantly higher than the amplitudes of other states. Thus combined, when we measure the state after application of Hadamard gates, we get the states $|101001\rangle$ or $|010110\rangle$ with probability $\approx (0.55)^2 + (0.55)^2 \approx 0.6$, which are precisely the Maximum for the defined problem, giving maximum objective function value of 6.

Hence we have showed explicitly how exactly the computations takes place using DQI.

B Intuition behind why this algorithm works **[Unfinished]**

Now let's discuss intuition behind DQI, why it works, and how one could have come up with it.

We have some function f and we want to find $\vec{x}$ such that $f(\vec{x})$ is maximum or atleast close to

maximum. Our initial thought would be to prepare something like the following state

$$|f\rangle = \frac{1}{\sqrt{c}} \sum_{\vec{x}} f(\vec{x}) |\vec{x}\rangle ,$$

where c is normalization constant. We know that probability of getting $\vec{x}$ when measuring the state above is

$$\text{prob}(\vec{x}) = \frac{|f(\vec{x})|^2}{c} .$$

We can notice that probability of producing $\vec{x}$ is more if $|f(\vec{x})|$ is higher. So this procedure gives us the ability to bias towards $\vec{x}$ s that maximize or minimize f .

Ideally, we would like quantum amplitudes to interfere destructively where value of f is lower and interfere constructively where value of f is higher. The state $|f\rangle$ doesn't give us a lot room to play with. Let's see what happens when we apply n -hadamards to $|f\rangle$.

$$|\tilde{f}\rangle = \frac{1}{\sqrt{c} \cdot \sqrt{2^n}} \sum_{\vec{x}, \vec{y}} (-1)^{\vec{x} \cdot \vec{y}} f(\vec{x}) |\vec{y}\rangle$$

We wanted to prepare $|f\rangle$. But if we can prepare $|\tilde{f}\rangle$, we can get $|f\rangle$. Now, if we had

$$f(\vec{x}) = (-1)^{\vec{b} \cdot \vec{x}} ,$$

then $|\tilde{f}\rangle$ would just be $|\vec{b}\rangle$. But now f are all just ± 1 , so measuring $|f\rangle$ will just give all bit strings with equal probability.

So $|\vec{b}\rangle$ allowed us to analyze f . What if we do the same but now with superposition of $\vec{b}_i$ s? If we start with $\frac{1}{\sqrt{2}}(|b_1\rangle + |b_2\rangle)$, then we can produce $|f\rangle$ with

$$f(\vec{x}) = (-1)^{\vec{b}_1 \cdot \vec{x}} + (-1)^{\vec{b}_2 \cdot \vec{x}} .$$

Now this does not have equal amplitudes anymore. When $\vec{b}_1 \cdot \vec{x} = \vec{b}_2 \cdot \vec{x}$, amplitude add up and when it is not, they cancel.

We can make one more observation here that we can change the values of which amplitudes cancel and which add by preparing $\frac{1}{\sqrt{2}}((-1)^{v_1} |b_1\rangle + (-1)^{v_2} |b_2\rangle)$ instead of $\frac{1}{\sqrt{2}}(|b_1\rangle + |b_2\rangle)$. Now the objective function is

$$f(\vec{x}) = (-1)^{v_1 + \vec{b}_1 \cdot \vec{x}} + (-1)^{v_2 + \vec{b}_2 \cdot \vec{x}} .$$

We can generalize and if we can prepare the state

$$\frac{1}{\sqrt{m}} \sum_{j=1}^m (-1)^{v_j} |\vec{b}_j\rangle ,$$

then we can produce $|f\rangle$ with

$$f(\vec{x}) = \sum_{j=1}^m (-1)^{v_j + \vec{b}_j \cdot \vec{x}}$$

This f gives a nice interpretation that you have m linear equations,

$$\vec{b}_j \cdot \vec{x} = v_j$$

and $f(\vec{x})$ is directly proportional to number of equations out of m simultaneously satisfied by that particular $\vec{x}$. This is exactly the problem max-XORSAT which we have been talking about.

Now, instead of producing $|f\rangle$, if we can produce $|f^2\rangle$,

$$|f^2\rangle = \frac{1}{\sqrt{c_2}} \sum_{\vec{x}} f(\vec{x})^2 |\vec{x}\rangle ,$$

then result will be $\vec{x}$ with probability proportional to $|f(\vec{x})|^4$
In this case, doing some algebra, we get

$$f(\vec{x})^2 = m + \sum_{j_1, j_2 \neq 1}^m (-1)^{v_{j_1} + v_{j_2} + \sum_{i=1}^n (\vec{b}_{j_1} + \vec{b}_{j_2}) \cdot \vec{x}}$$

Now, we can calculate which state, if we apply n -Hadamard to, gives us $|f^2\rangle$. That state is

$$|\tilde{f}^2\rangle = \frac{1}{\sqrt{c_2} \sqrt{2^n}} \sum_{\vec{y}} f(\vec{x})^2 (-1)^{\vec{x} \cdot \vec{y}} |\vec{y}\rangle .$$

Unfortunately, due to time constraint, I could not finish this section.